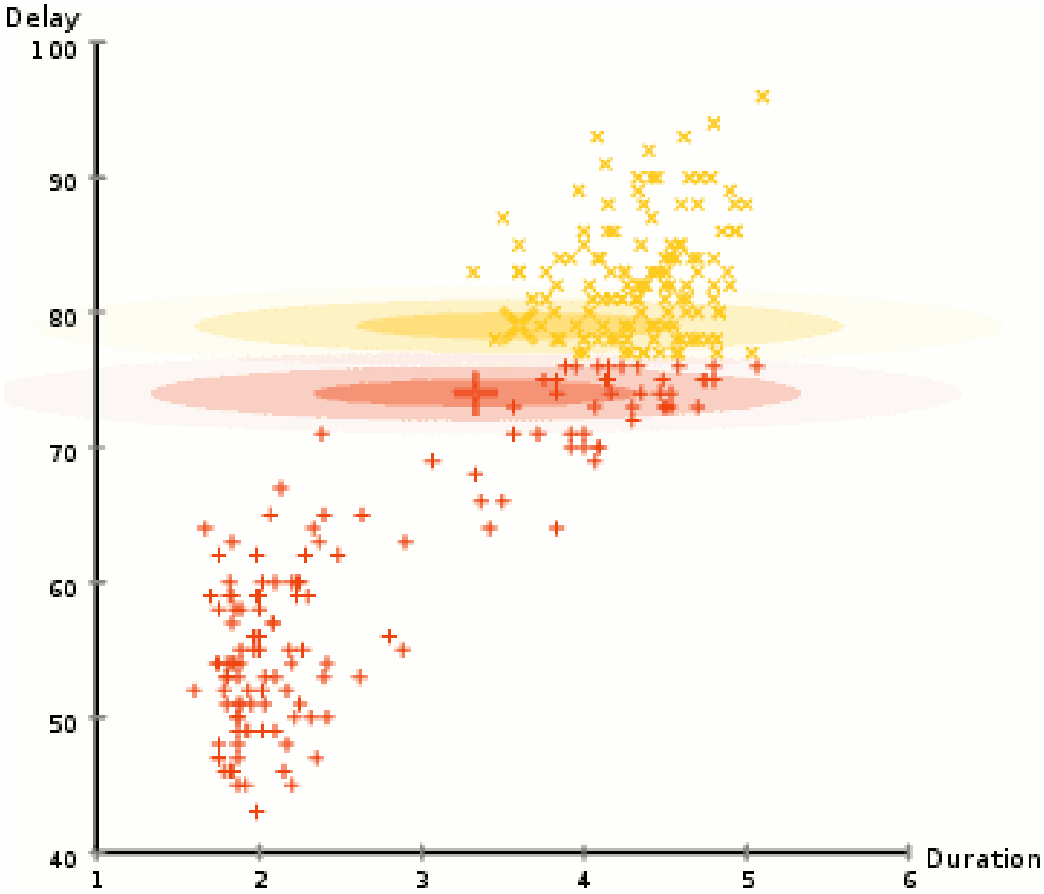
DA5400| BE22B048

Report: Assignment 2

EM & Linear Regression



Files Used

Data Files : A2Q1, A2Q2Data_test - A2Q2Data_test, A2Q2Data_train - A2Q2Data_train

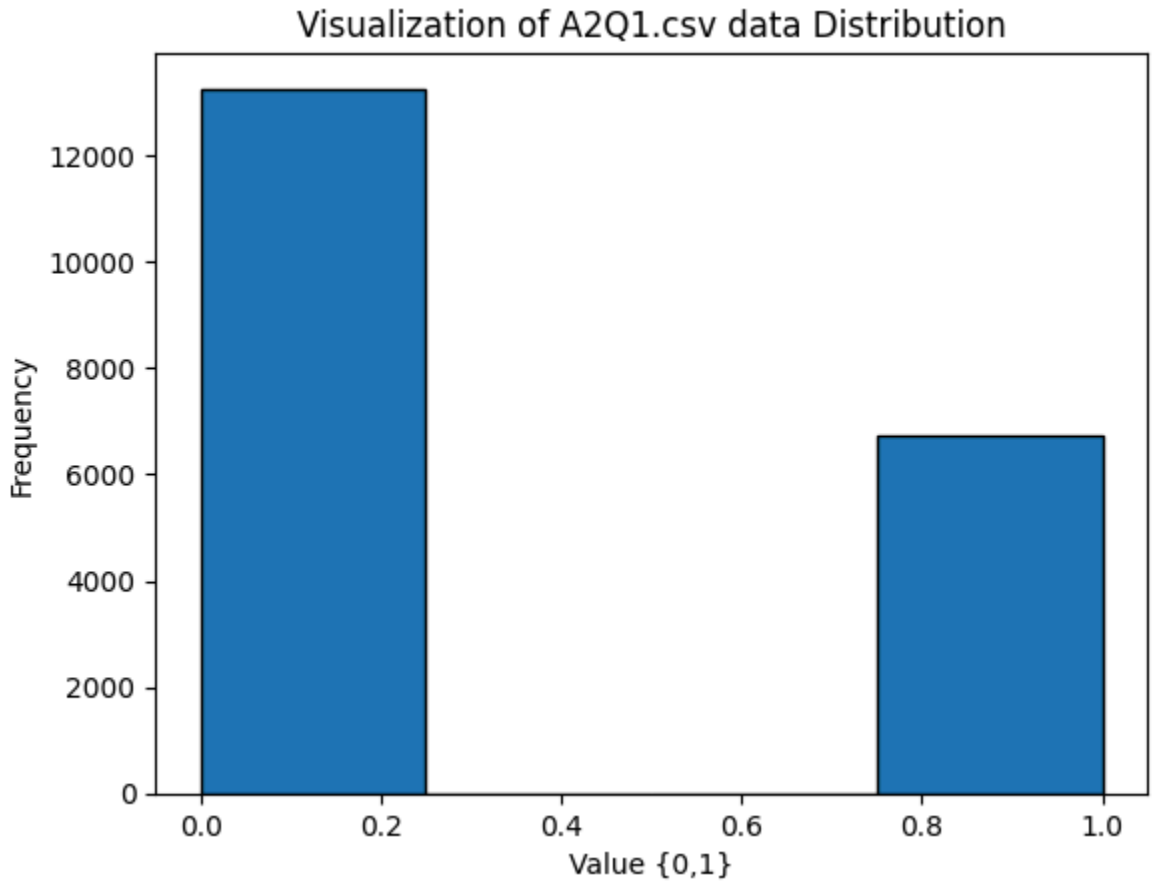
Code Files: question1, question2

Problem 1: Estimation Maximisation

Part 1: Choosing a Probabilistic Model for the Data

The data comprises 400 rows where each feature is a binary value, either 0 or 1.

I am taking the data to be in an N by D format, where N represents the number of datapoints and D is the number of features.



Thus, given the binary nature of the data, a Mixture of Bernoulli distributions is the most appropriate choice to model it.

The following are my assumptions regarding the generating model:

There are K 4 different multivariate Bernoulli distributions

Each of these K distributions (or "clusters") has its own set of parameters, and each data point is drawn from one of these clusters.

A single multivariate Bernoulli distribution for a D-dimensional data point.

$$\mathbf{x} = (x_1, \ldots, x_D)$$

is defined by a parameter vector

$$\theta = (\theta_1, \ldots, \theta_D)$$

Where $theta_d$ is the probability that the dth feature x_d is 1.

The probability of observing a data point x is:

$$P(\mathbf{x}|\theta) = \prod_{d=1}^{D} \theta_d^{x_d}(1-\theta_d)^{1-x_d}$$

The prior Probability of a data point belonging to any one cluster is given by π_k. Since π_k is a probability value. The sum of all prior probabilities for a point is 1.

$$\sum_{k=1}^{K} \pi_k = 1.$$

Derivation of the EM algorithm.

Assumptions

- I.I.D. Data:

  The data points are independent and identically distributed (i.i.d.). Thus, we can write the total likelihood as a product of likelihoods.

- Mixture Generation:
  Each data point is generated by first choosing a single cluster k with probability π_k and then drawing from that cluster's specific distribution.

- Conditional Independence:
  Within a given cluster k, all D features are assumed to be independent. Thus, the probability of x is a product of its individual features.

I Likelyhood of a Single Point

$$p(x_n) = \sum_{k=1}^{K} p(\text{cluster } k)\, p(x \mid \text{cluster } k)$$

$$p(x_n \mid \pi, \theta) = \sum_{i=1}^{K} \pi_k\, p(x_n \mid \theta_k) \quad (1)$$

$$p(x_n \mid \theta_k) = \prod_{d=1}^{D} p(x_{nd} \mid \theta_{kd})$$

$$= \prod_{d=1}^{D} \theta_{kd}^{x_{nd}} (1 - \theta_{kd})^{1 - x_{nd}}$$

(1) Becomes

$$p(x_n \mid \pi, \theta) = \sum_{i=1}^{1} \pi_k \left(\prod_{i=1}^{D} \theta_{kd}^{x_{nd}} (1 - \theta_{kd})^{1 - x_{nd}} \right) \quad (2)$$

II Likelyhood for the entire dataset

By iid assumpth

$$L(\pi, \theta \mid X) = \prod_{i=1}^{N} p(x_n \mid \pi, \theta)$$

Replace $p(x_n \mid \pi, \theta)$ from (2)

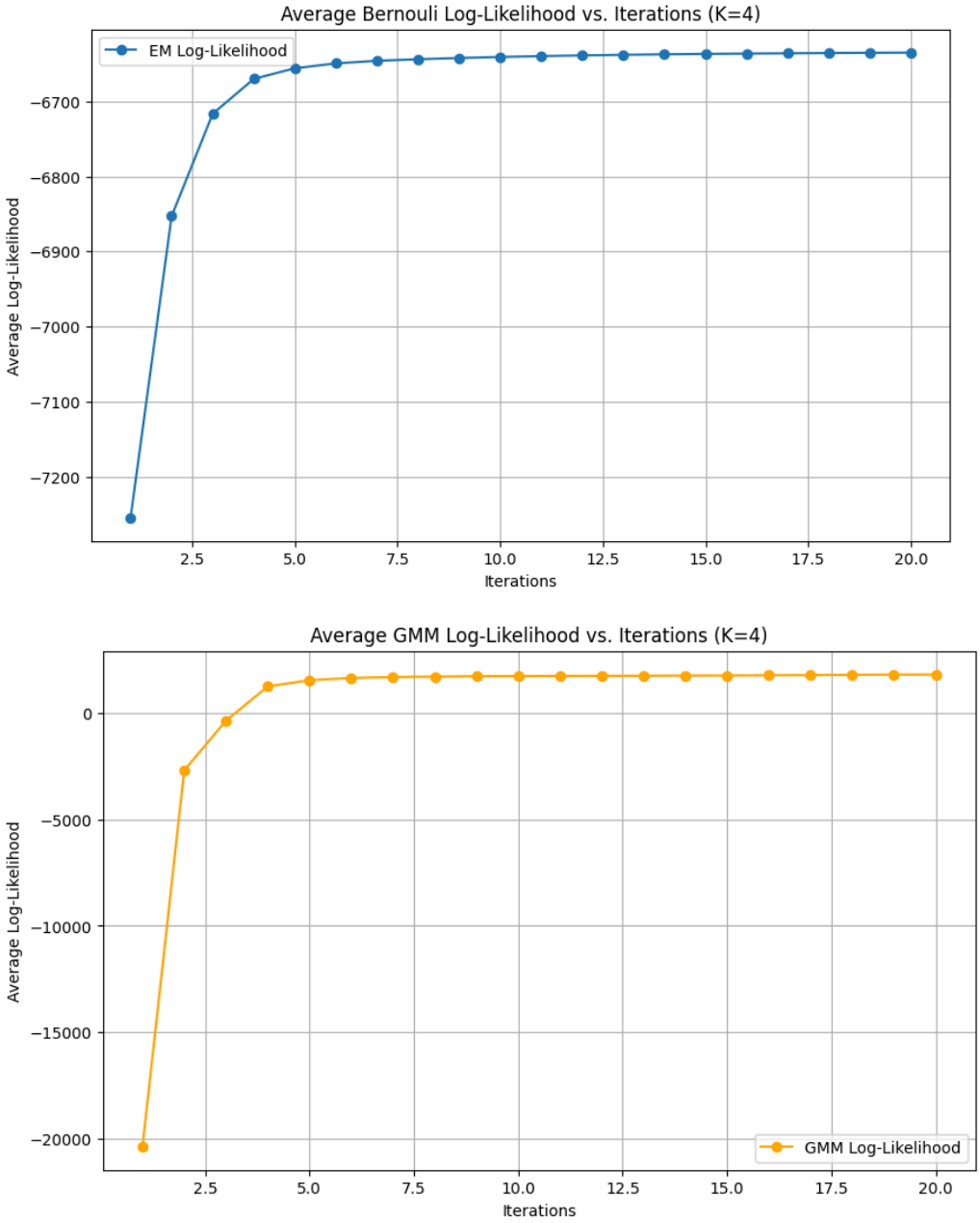
$$L(\pi, \theta \mid X) = \prod_{i=1}^{N} \left[\sum_{k=1}^{K} \pi_k \prod_{d=1}^{D} \theta_{kd}^{x_{nd}} (1-\theta_{kd})^{1-x_{nd}} \right]$$

III Log likelyhood Expression ($\mathcal{L}$)

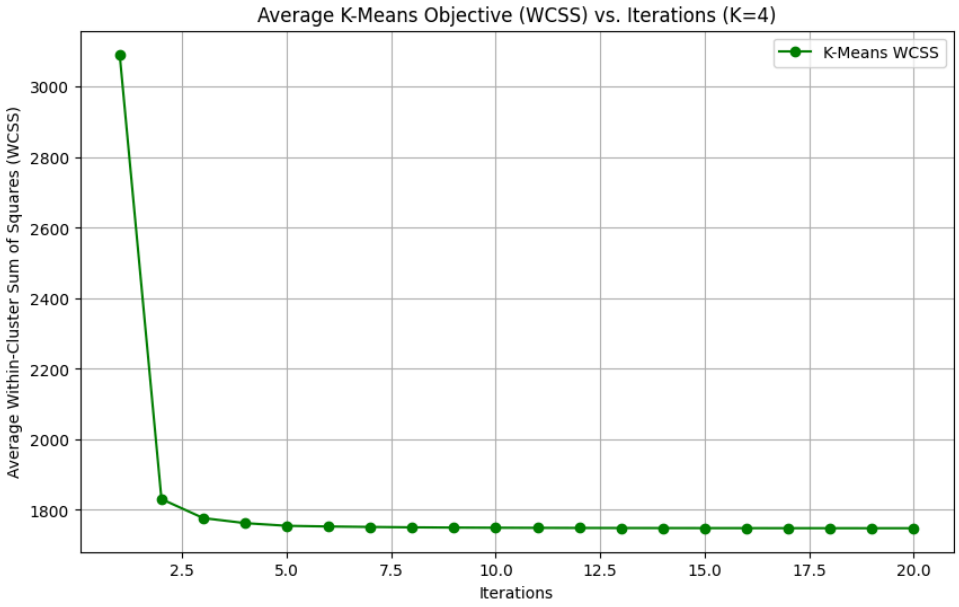
$$\mathcal{L} = \log \left(\prod_{i=1}^{N} \left[\sum_{k=1}^{K} \pi_k \prod_{d=1}^{D} \theta_{kd}^{x_{nd}} (1-\theta_{kd})^{1-x_{nd}} \right] \right)$$

$$= \sum_{n=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \prod_{d=1}^{D} \theta_{kd}^{x_{nd}} (1-\theta_{kd})^{1-x_{nd}} \right)$$

Plots for Convergence of Algorithms



On average, both the Bernoulli Mixture model and the GMM converge within 10 iterations.



Similarly, the K-means, within-cluster sum of squares stabilises at a fixed value after 10 iterations, showing that it too is convergent.

Choice of Optimal Model

Bernoulli Mixture Model:

Assumption: The data is generated from a mixture of multivariate Bernoulli distributions.

- Data Fit: This model assumes that each feature is a binary (0 or 1) event, and each cluster has a specific probability.
- Appropriateness:
    - This is the most appropriate model for this dataset.
    - The model's core assumption perfectly matches the data's binary nature. The parameters pi and theta are directly interpretable as "the probability of belonging to cluster ‘k" and "the probability of the dth feature being 1 in the kth cluster.

Gaussian Mixture Model (GMM):

- Assumption: The data is generated from a mixture of multivariate Gaussian (continuous) distributions.
- Data Fit: This model is designed for continuous data that can take any real value.
- **Appropriateness**:

  - This is a poor choice for this dataset. Forcing a GMM onto binary data is a significant model mismatch.

  - The GMM will try to find "means" and "covariances" for data that only exists at 0 and 1.

  - While the algorithm runs and the log-likelihood will converge (as seen in the plot above), the underlying parameters are not meaningful.

  - The mean mu will be a vector of values between 0 and 1, but interpreting it as the centre of a "bell curve" in a binary space is incorrect.

  - Most importantly, this model will generate points that simply cannot be found in this dataset!

K-Means:

- **Assumption**: This is a geometric, not a probabilistic, algorithm. It assumes clusters are spherical and that Euclidean distance is a meaningful way to measure similarity.
- **Data Fit**: K-Means minimises the sum of squared Euclidean distances
- **Appropriateness**:
  - While K-Means is often used for clustering, its objective function (WCSS) is questionable for binary data. The squared Euclidean distance between two binary vectors is equivalent to the Hamming distance (the number of bits that differ).

- However, the "centroid", mu, will be a vector of real numbers between 0 and 1 (the mean of the binary vectors in the cluster), which can NEVER be a point in the original dataset. This makes the centroid hard to interpret.

The Best Model

The fundamental principle of good modelling is to choose a model whose assumptions align with the data. Thus, the Bernoulli mixture model is the best for the reasons detailed below.

- The `A2Q1.csv` dataset is composed of binary vectors.
- The Bernoulli Mixture Model is explicitly designed for mixtures of binary data.
- The GMM and K-Means algorithms are designed for continuous data and rely on assumptions (Gaussian distributions, Euclidean space) that are violated by this dataset.

Problem 1: Estimation Maximisation

Analytical Solution

The analytical, i.e. least squares solution was computed using the Normal Equation.

```
w = (X^T * X)^-1 * (X^T * y)
```

The derivation is given below.

Analytical Solution

loss funct^{n}: $J(w) = \|Xw - y\|^2$

$X \in \mathbb{R}^{N \times D+1}$ $\quad y \in \mathbb{R}^{N \times 1}$

$$\nabla_w J = \nabla \left((Xw - y)^T (Xw - y) \right)$$

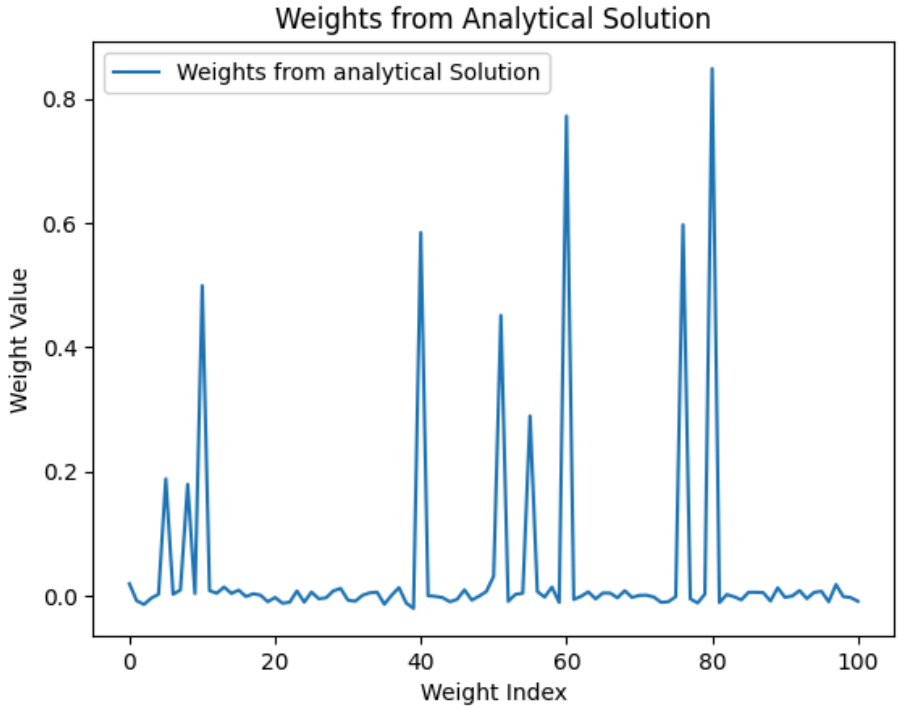
$$= \nabla \left(w^T X^T X w - 2 w^T X^T y + y^T y \right)$$

$$= 2 X^T X w - 2 X^T y$$

Assuming $X^T X$ is invertible

Setting $\nabla_w J$ to 0 gives

$$w = (X^T X)^{-1} X^T y$$



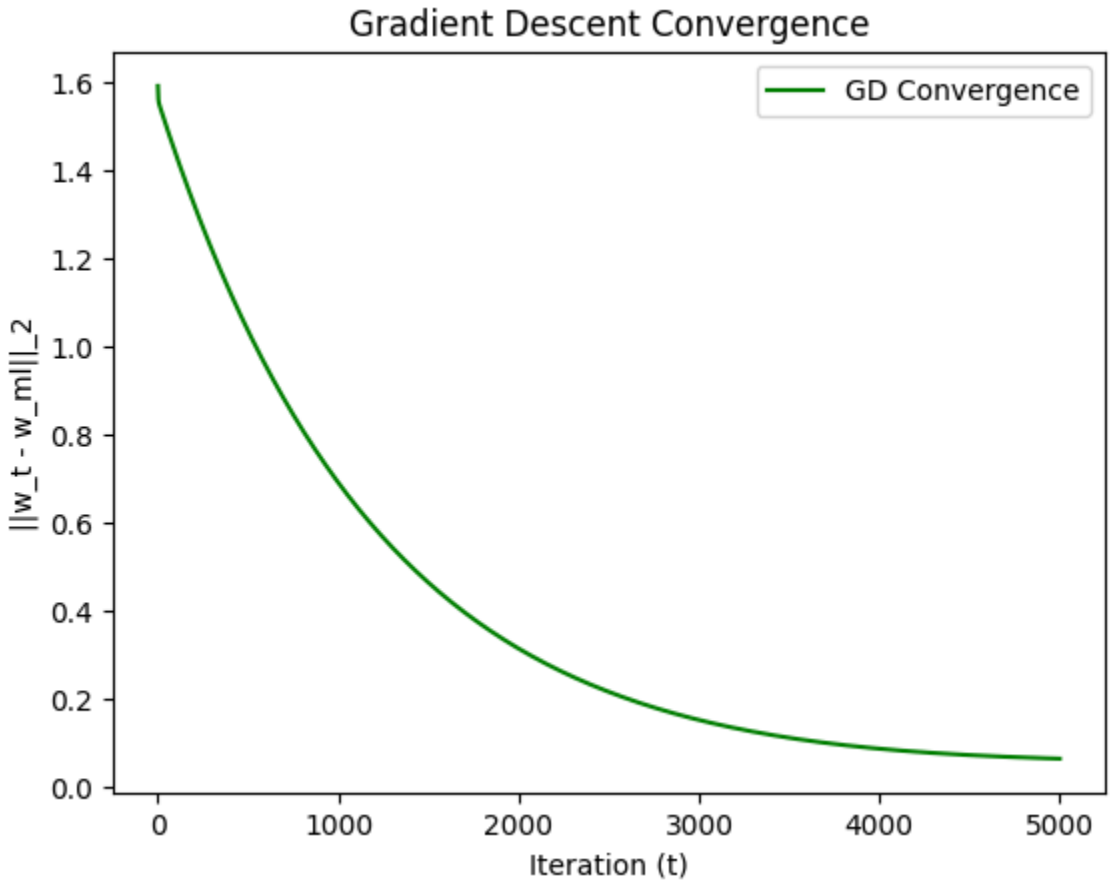
Performance of the analytical solution

- Training MSE (Analytical): 0.03968521086820023
- Test MSE (Analytical): 0.37075150228804377

Convergence of the Gradient Descent (GD) algorithm

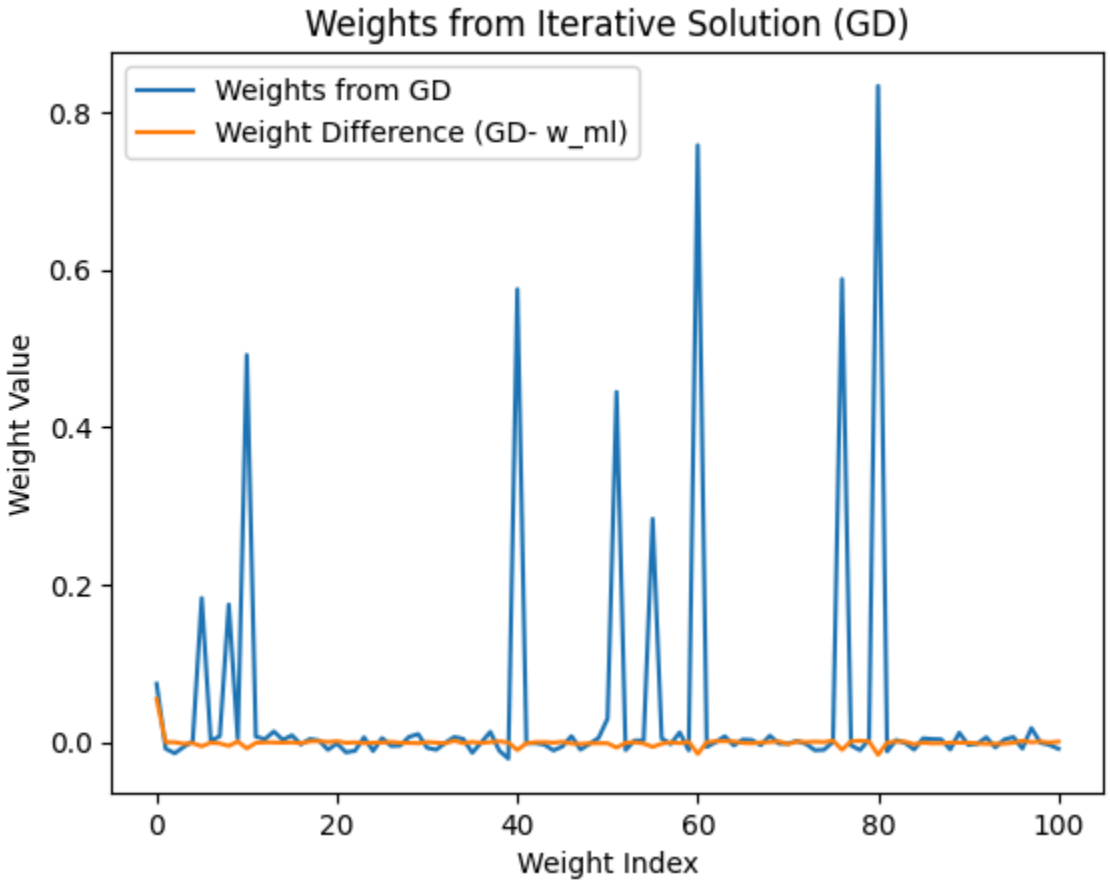
The plot below shows:

1. The distance between the iterative solution and the true solution decreases at each iteration. The decrease is monotonic!
2. The distance decreases very quickly in the initial iterations and then slows down, approaching zero asymptotically.
3. If the learning rate, eta, is too large, the norm might oscillate or even diverge (increase). However, 0.01 shows stable convergence.



We observe that the algorithm converges, albeit it requires a small learning rate and many iterations (around 5000) to converge to the least squares solution.

A learning rate lower than 0.01 is necessary to prevent the problem of exploding gradients, where gradients grow exponentially, resulting in large weight updates and unstable training.



Here, the difference vector between the analytical solution and GD is due to the number of iterations being less than required due to the lack of computational power on my system. It will converge to the analytic solution if the number of iterations is increased.

Performance

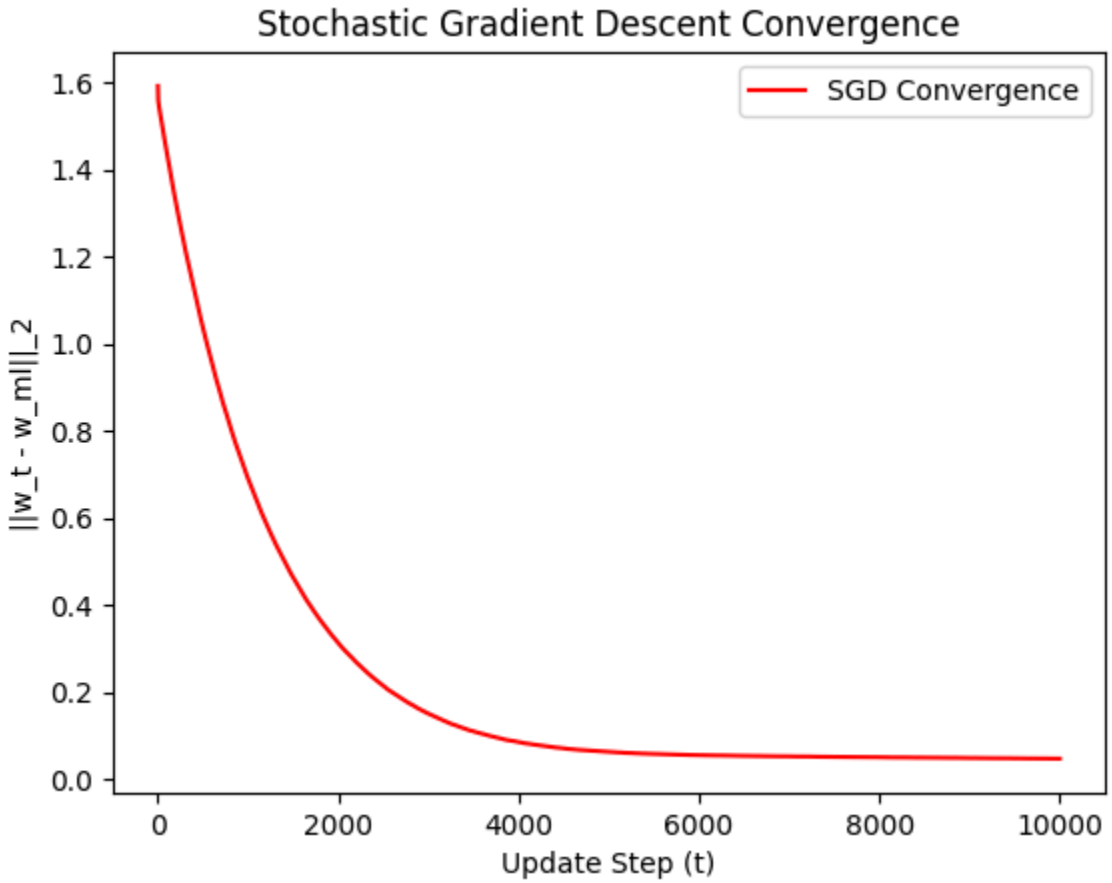
- Training MSE (GD): 0.04726099989316089
- Test MSE (GD): 0.3110705125630293

Stochastic (Mini Batch) Gradient Descent

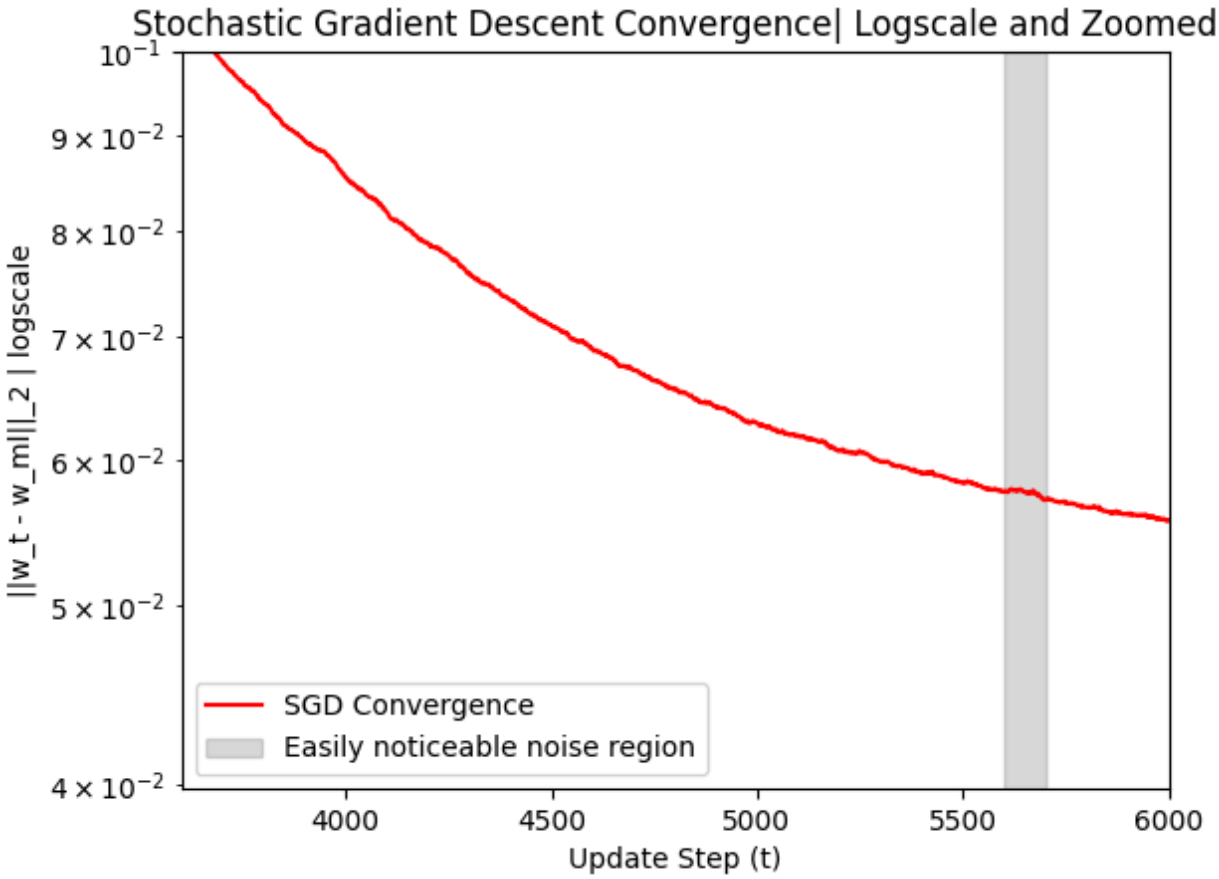
This works similarly to GD. However, instead of using the entire dataset to compute the gradient at each step, we use a small, random "mini-batch" of data.

The plot below shows:

1. The distance between the iterative solution and the true solution has a general decreasing trend. However, it is noisy and not a monotonic decrease! The noisy nature of the plot is hard to view at this scale and is easier to spot in the logscale plot.
2. The distance decreases very quickly in the initial iterations and then slows down, approaching zero asymptotically. The convergence is faster (in terms of wall-clock time) as SGD performs more computations per epoch due to sampling.

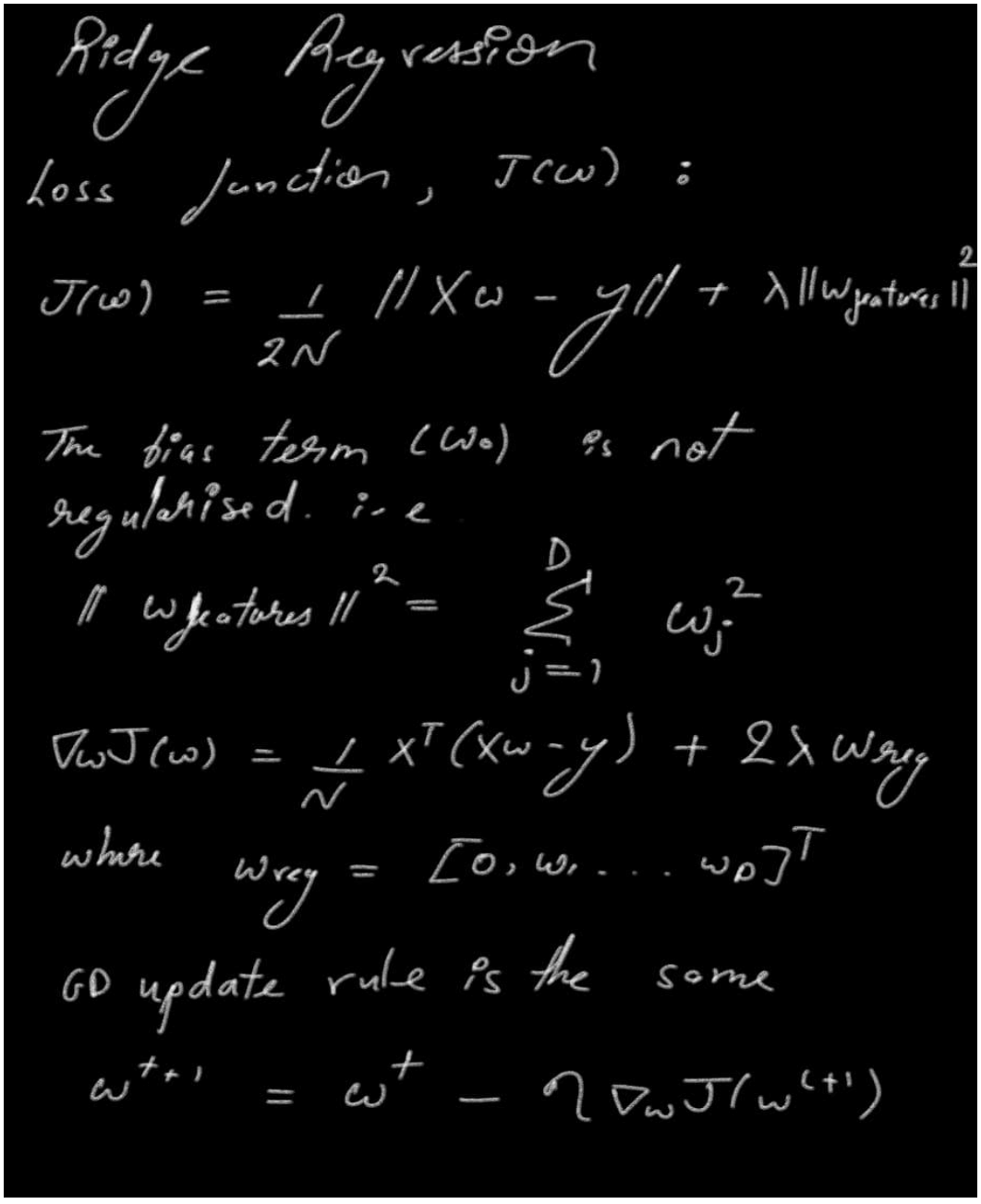


The zoomed-in log scale illustrates the noise present in the convergence of Stochastic Gradient Descent (SGD). Although the norm tends to decrease on average, it exhibits fluctuations at each step. This variability arises because each mini-batch offers only a rough, "stochastic" estimate of the true gradient. Nevertheless, the gradient estimates obtained from a subset of the data points are generally quite accurate, resulting in minimal noise.



It does not converge to exactly the analytical solution, as the number of iterations is not sufficient. Providing enough computational power and using a learning rate scheduler for efficiency will ensure it converges optimally.

Ridge Regression

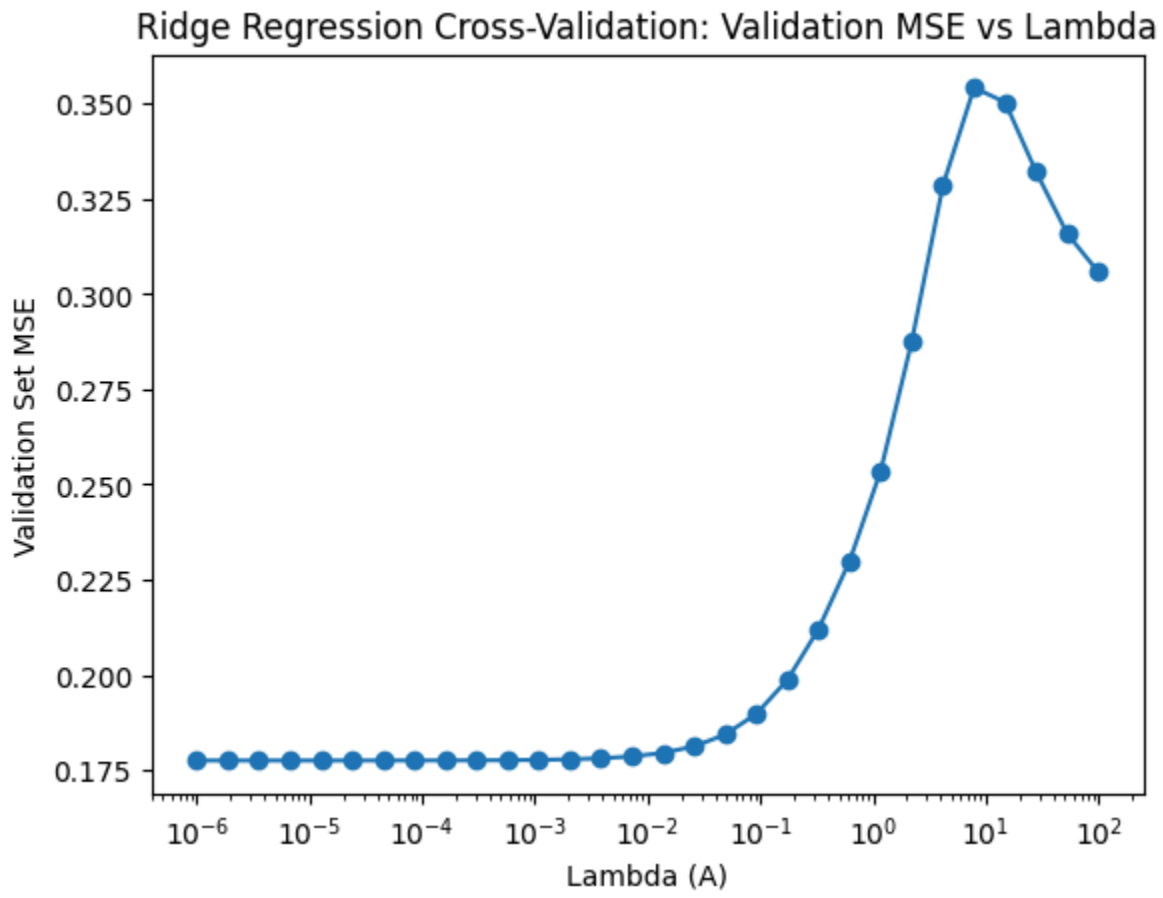


Code for the Ridge Regression does the following:

- Implements the Ridge GD algorithm.
- Splits the *training* data into a sub-training set and a validation set.
- Train a Ridge model for different lambda values on the sub-training set.
- Evaluates each model on the validation set to find the best model
- Train a final w_R on the *full* training set using the best lambda

Compare the test error of w_{ml} from Part I and w_R on the held-out test data.

MSE vs Lambda Plot



The data are in a "low-dimensional" regime, and we have 100 samples for every feature. This abundance of data makes the least-squares problem well-determined. W_{ml} is already a stable solution with low variance.

Ridge Regression's main job is to reduce variance by adding a little bias. Since the analytical model already had low variance, there was no real problem for the lambda penalty to solve. Adding any significant regularisation lambda only added unnecessary bias, which increased the error.

However, in line with the theorem, having a small value of lambda leads to lower test error.

Thus, the ridge model is better than the analytic model

Why the Ridge model is better

```
Best lambda (A) found: 1e-06
Validation MSE at best lambda: 0.17754210098485795
Final Ridge model (w_R) trained with 2000 iterations.

--- Test Error Comparison ---
Test MSE for w_ML (Least Squares): 0.37075150228804377
Test MSE for w_R (Ridge Regression): 0.25576538814887106
Observation: The Ridge Regression model (w_R) performed better on the test set.
```

The ridge model achieves a 25% test error as compared to the analytic solution, which achieves 37% test error.

This can be explained by the bias-variance tradeoff.

$$E[(y_{test} - \hat{y}_{test})^2] = \text{Bias}(\hat{w})^2 + \text{Variance}(\hat{w}) + \sigma^2 \text{ (irreducible noise)}$$

Ridge Regression aims to decrease the variance by a large amount while increasing the bias by a small amount. Ridge works well when the **reduction in Variance is greater than the increase in Bias-squared**. In high-dimensional problems, the benefit of reducing the variance is so high that this tradeoff is almost always a win.

Analytic Solution

$$w_{ML} = (X^T X)^{-1} X^T y$$

$$\text{Bias} = 0$$

$$\text{Variance} \propto (X^T X)^{-1}$$

Variance can be very high if $X^T X$ becomes ill-conditioned or near singular, i.e has a nearly 0 determinant.

This will cause $(X^T X)^{-1}$ to have large values.

The model will be hypersensitive to training data

$X^T X$ is ill conditioned if:

(i) Features are correlated.

(ii) $N \approx D$

Analytic Solution

$w_{ML} = (X^T X)^{-1} X^T y$

Bias $= 0$

Variance $\propto (X^T X)^{-1}$

Variance can be very high if $X^T X$ becomes ill-conditioned or near singular, i.e has a nearly 0 determinant.

This will cause $(X^T X)^{-1}$ to have large values.

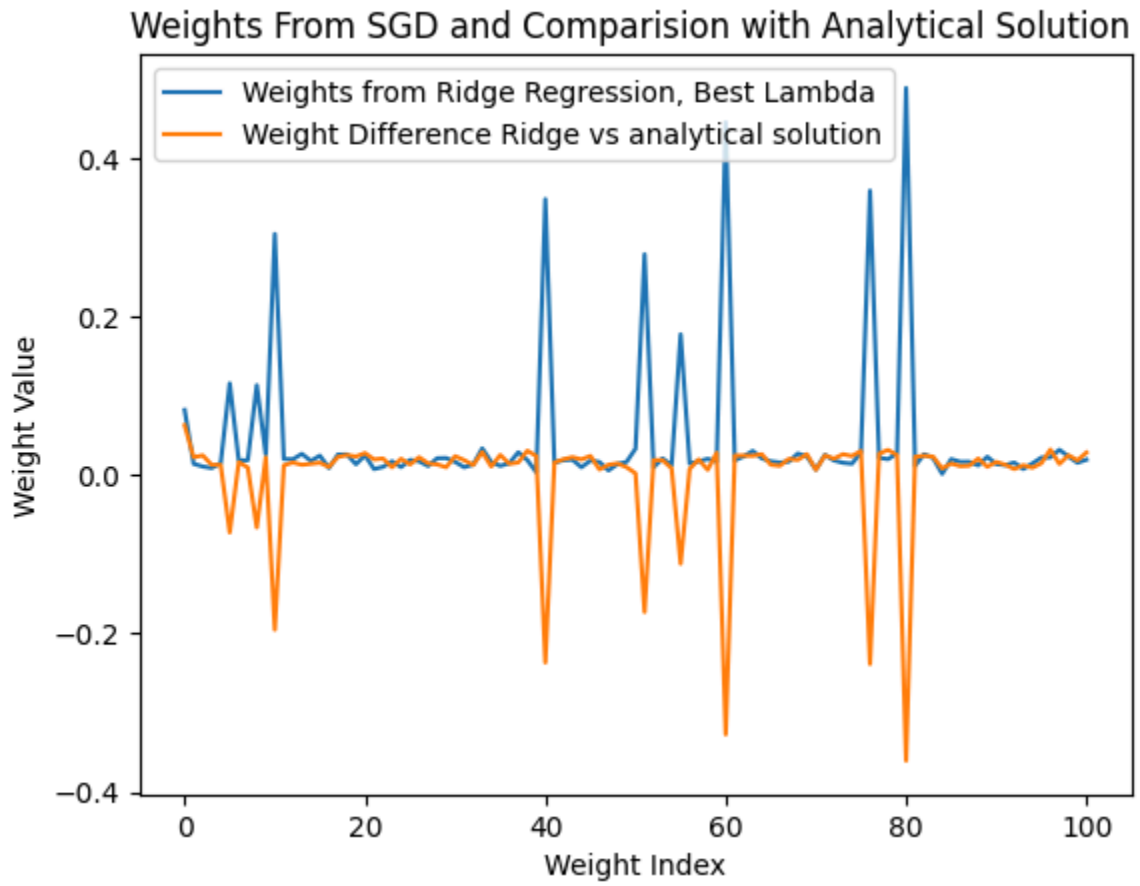
The model will be hypersensitive to training data

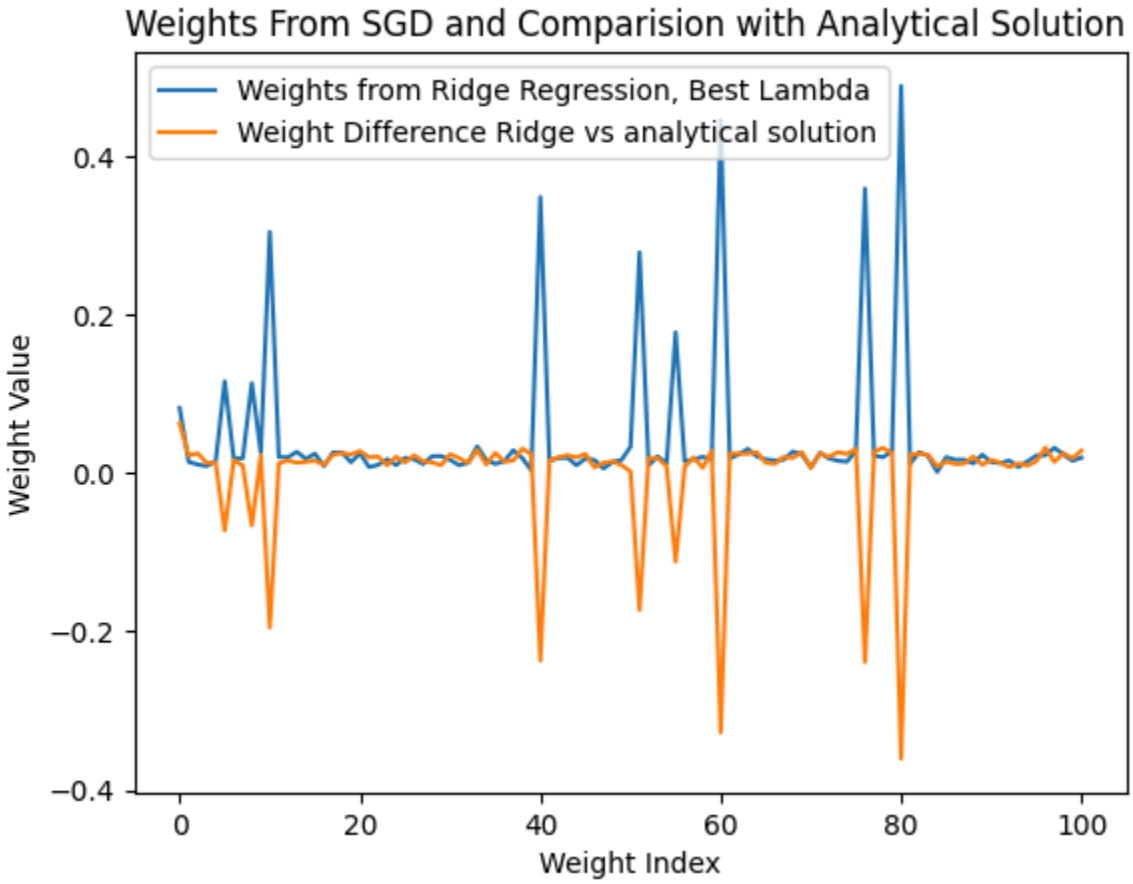
$X^T X$ is ill conditioned if:

(i) Features are correlated.

(ii) $N \approx D$

As expected, the magnitude of the weights is much less in the Ridge method.




Weights From SGD and Comparision with Analytical Solution
Weights from Ridge Regression, Best Lambda
Weight Difference Ridge vs analytical solution
Weight Value
Weight Index
0.4
0.2
0.0
−0.2
−0.4
0
20
40
60
80
100